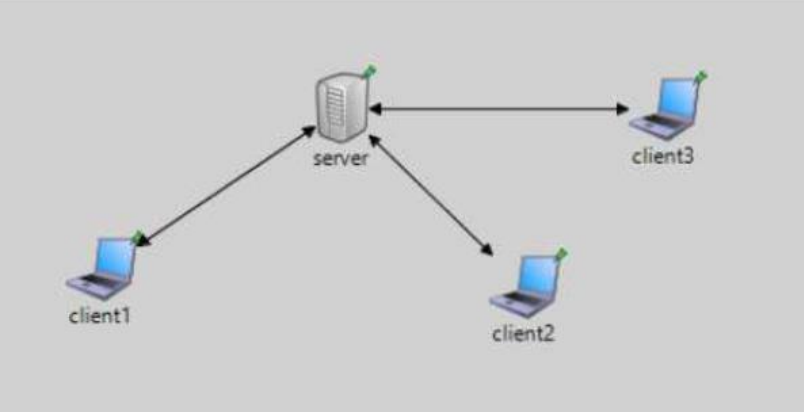
TP N°4
Communication synchrone et asynchrone dans les systèmes distribués

Description

Le but du TP est d'implémenter un système de communication synchrone et asynchrone entre un serveur et 3 clients. L'architecture du système est la suivante :



Partie 1 Communication synchrone

Les clients envoient des requêtes de calcul pour l'addition de deux entiers aléatoires a et b au serveur et se bloquent en attente de la réponse. Chaque client doit afficher les messages suivants :

- Le client i envoi une requête pour le calcul de l'addition de a et b
- Le client i est bloqué en attente de la réponse

Le serveur traite les messages des clients en effectuant l'addition de a et b puis, envoi les réponses aux clients. Le serveur doit afficher les messages suivants :

- Réception de la requête du client i, traitement….
- Envoi de la réponse au client i

Lorsque la réponse est reçue par le client, celui-ci se débloque et affiche les messages suivants à l'écran :

- Le client i est débloqué - réponse reçue du serveur avec le résultat r

Le client peut envoyer une nouvelle requête au serveur.

TP N°4
Communication synchrone et asynchrone dans les systèmes distribués

Partie 2 Communication asynchrone

Les clients envoient des requêtes de calcul pour l'addition de deux entiers aléatoires a et b au serveur chaque période de temps d (passé en paramètre). Chaque client doit afficher les messages suivants :

- Le client i envoi une requête pour le calcul de l'addition de a et b

Le serveur traite les messages des clients en effectuant l'addition de a et b puis, envoi les réponses aux clients. Le serveur doit afficher les messages suivants :

- Réception de la requête du client i, traitement….
- Envoi de la réponse au client i

Lorsque la réponse est reçue par le client, celui-ci affiche les messages suivants à l'écran :

- Réponse reçue du serveur avec le résultat r

Notes importantes :

- A la fin de la simulation, on voudrait afficher les statistiques suivantes :
  - Pour chaque client, le nombre de requêtes envoyées au serveur et le nombre de réponses reçues du serveur.
  - Pour le serveur, le nombre moyen de requêtes traitée par client.
- Les messages au niveau du serveur sont traités en ordre FIFO de réception.
- Dans un réseau réel, l'ordre de réception des messages au niveau du serveur est aléatoire. Pour simuler cette situation, il faudra utiliser la primitive sendDelayed qui envoi le message après une certaine période aléatoire. Syntaxe : sendDelayed (message , exponential(valeur), "nom-passerelle",k)